Escola d'Enginyeria de Telecomunicació i Aeroespacial de Castelldefels

UNIVERSITAT POLITÈCNICA DE CATALUNYA BARCELONATECH

Navegació Aèria, Cartografia i Cosmografia (NACC)

LAB. SESSION 1

GROUND TRACK OF GPS SATELLITES

Joan Olmos

EETAC 2025-26

V1.0

Contents

1 Objective 3

2 Understanding the GPS ephemeris 3

3 Main program 4

3.1 Step 1 5

3.2 Step 2 5

3.3 Step 3 6

3.4 Step 4 6

3.5 Step 5 7

3.6 Step 6 7

3.7 Step 7 8

3.8 Step 8 8

3.9 Step 9 9

4 Final remarks 9

5 References 10

1 Objective

The objective of this lab. work is to be able to plot the satellite ground track for any given satellite or constellation of satellites. Initially we will work with the GPS constellation because the real-time ephemeris are available in the Internet in a clear text format, but the programs can be applied to any satellite for which the Keplerian elements are known.

This statement will provide guidelines as well as the Matlab code structure that must be followed literally.

2 Understanding the GPS ephemeris

The GPS almanac is the set of satellite ephemeris that every GPS satellite transmits. It includes information about the status (health) of the entire GPS satellite constellation and the Keplerian elements of each satellite orbit. These files can be obtained from the source (1) or from http://celestrak.com/GPS/almanac/Yuma/2026/

The almanac contains the following data and Keplerians about each satellite (2):

1) **ID**: PRN (Pseudo Random Noise) id. of the SVN (Space Vehicle NAVSTAR)
2) **Health:** Indicates the operational state of the SV (Space Vehicle). The value "000" means that the SV is usable.
3) **Eccentricity:** This shows the amount of the orbit deviation from a circular orbit. It is the distance between the foci divided by the length of the semi-major axis (the NAVSTAR orbits are almost circular).
4) **Time of Applicability (ToA):** The time instant (seconds within the GPS week) for which the almanac has been computed (ephemeris epoch).
5) **Orbital Inclination:** The angle of the SV orbit plane with respect to the equator (GPS is at approx. 55 degrees) in radians. The SV ground track will not rise above approx. 55 degrees of latitude.
6) **Rate of Right Ascension:** Rate of change of the angle of right ascension in rad/s.
7) **SQRT(A) Square Root of Semi-Major Axis:** The semi-major axis is the distance from the center of the ellipse to either the point of apogee or the point of perigee.
8) **Right Ascension at GPS week epoch:** Geodetic longitude (in [rad] with respect to Greenwich meridian) of the ascending node of the orbit at the GPS week epoch (Saturday to Sunday midnight).
9) **Argument of Perigee at ToA:** An angular measurement [rad] along the orbital path measured from the ascending node to the point of perigee. It is measured in the direction of the SV's motion.
10) **Mean Anomaly at ToA:** Angle [rad] traveled past the perigee at ToA. When the SV has passed perigee and heading towards apogee, the mean anomaly is positive. After the point of apogee, the mean anomaly value will be negative to the point of perigee.
11) **Af(0): SV clock bias in seconds**
12) **Af(1): SV clock Drift in seconds per seconds**

13) Week: GPS week number (0000-1023). Counter increased (module 1024) every week since 0h UTC of Jan 6th, 1980 (the GPS epoch = 2444244.5 JD). First roll-out happened at 23:59:47 (UTC) on Aug 21, 1999 (roll epoch = 2451412.499850 JD).

3 Main program

This section describes the skeleton and code structure of the main program that you must write. Comments are as important as the code itself. Create a new folder dedicated to this lab. work and copy there all the files available in Atenea for this lab. work. Copy and paste the next text literally to a matlab script (call it “main1.m”) and leave room to write your own code in the places specified.

```
clear all
close all

G = 6.67384e-11;                            % Gravitational constant
Me = 5.972e+24;                             % Earth Mass
Oe = 7.2921151467e-5;                       % Earth rotation angular velocity (rad/s)
tol = 1.e-8;                               % tolerance for convergence

global fd;
fd = fopen("main1.dat", "w");

plotworld();

esec = gpstime(DD, MM, YYYY, hh, mm, ss);
% esec = gpstime();

% ******** Week 334 almanac for PRN-03 ********
% ID:                         03
% Health:                     000
% Eccentricity:               0.6413459778E-002
% Time of Applicability(s):  405504.0000
% Orbital Inclination(rad):   0.9908221593
% Rate of Right Ascen(r/s):  -0.7966046104E-008
% SQRT(A)   (m 1/2):          5153.672363
% Right Ascen at Week(rad):   0.1092726121E+001
% Argument of Perigee(rad):   1.228802126
% Mean Anom(rad):            -0.1791370106E+001
% Af0(s):                     0.6790161133E-003
% Af1(s/s):                  -0.3637978807E-011
% week:                        334

id = 3;                        % sat. identifier
e = 0.6413459778E-002;         % orbit eccentricity
toa = 405504;                  % Time of Applicability [seconds within current week]
io = 0.9908221593;             % orbit inclination [rad]
dO = -0.7966046104E-008;       % derivative of right ascension of AN [rad/s]
a = 5153.672363^2;             % orbit semi-major axis [m]
Lo = 0.1092726121E+001;        % longitude of AN at week epoch [rad] (uncorrected)
w = 1.228802126;               % argument of perigee [rad]
Mo = -0.1791370106E+001;       % Mean Anomaly at ToA [rad]

===== your code goes here ====

fprintf(fd, "Oo = %f, esec = %f, dt = %f\n\n", mod(Oo, 2 * pi), esec, dt);

===== your code goes here ====

plot(long, lat, 'r*')
text(long + 2, lat + 1, sprintf('%d', id));

===== your code goes here ====

fclose(fd);
```

The first variables are constants that will be used through the code development.

The file descriptor “fd”, once opened, will create a file for debugging and results comparison.

The Matlab function “plotworld()” is provided in Atenea and will plot a map of the world so that your code can plot your results on top of it.

The Matlab function “gpstime(DD, MM, YYYY, hh, mm, ss)” is also provided and will return the variable “esec”, which contains the number of seconds elapsed from week epoch (last Saturday to Sunday midnight, i.e., 00h:00m:00s of last Sunday) until the date and time (DD, MM, YYYY, hh, mm, ss) passed as the argument. You should set that “target” date equal to the UTC time at which you are computing the satellite position. It could be the date and time of start of this project (for example). Notice that your computer clock is ahead of UTC by 1 hour in winter and by 2 hours in summer time.

The “gpstime.m” function will also download from the Celestrak web page the almanac file that shows a ToA as close as possible to the given date.

Calling the “gpstime()” function without arguments will return the “esec” for NOW, where NOW means the time at which the matlab script is run. So calling the “gpstime()” function without arguments will always return a different “esec”.

Since the online tool that we will use for comparison, https://www.nstb.tc.faa.gov/rt_waassatellitestatus.htm, only gives real time GPS satellite positions you should make a screen capture of that URL at the target date (DD, MM, YYYY, hh, mm, ss) of starting this project and save it for later on.

If you call “gpstime()” without arguments your “esec” variable and your results will always be in real time, but since the sat. ephemeris have been “hard coded” at “main1.m” you should manually update the ephemeris data each time that “gpstime()” downloads a more recent almanac, otherwise the obtained results may begin to slightly depart from reality after several days. Celestrak publishes a new almanac every day of the week.

Once you have called “gpstime.m”, copy from the downloaded almanac file the ephemeris of the GPS satellite that has an ID equal to your lab. group number and paste it in your “main1.m“ file overwriting the given example for “PRN-03”. Comment out that section (just like in the previous example).

Next assign a name and a value (copy and paste) to the variables containing the satellite ephemeris. Do it just like in the previous example (using the same variable names) and then proceed to complete orderly the next steps:

3.1 Step 1

Before the first “fprintf()” statement in “main1.m” you must compute the variable “dt”, which is the time elapsed (in seconds) from the ToA of the ephemeris until the target date and time (DD, MM, YYYY, hh, mm, ss) for computing the sat. position. “dt” must be negative if the target date is before the ToA.

Also at this moment you must compute the longitude of the satellite Ascending Node (AN) at the ToA in radians. Just look at the meaning of the “Lo” variable and consider that the AN of any orbit remains fixed at a point on the celestial equator (it doesn’t turn with the Earth) and so the longitude of the AN for any satellite DECREASES proportionally to the elapsed time at a rate of Oe [rad/s].

Just call “Oo” to the resulting longitude of the satellite Ascending Node (AN) at ToA in radians.

After this the “main1.m” code will print “Oo”, “esec” and “dt” to the debugging file, so that you can compare your results during the class of next Thursday.

3.2 Step 2

Write a matlab script (following the given prototype) to compute the current Mean Anomaly (in radians) of the satellite.

```
% Compute the Mean Anomaly at "dt" seconds after ToA, given the Mean Anomaly at ToA and
% the semi-major axis
% G = gravitational constant
```

```
% Me = Earth mass
% a = semi-major axis [m]
% Mo = Mean Anomaly at ToA [rad]
% dt = seconds elapsed after ToA (negative if ToA is in the future)
% M = Mean Anomaly at ToA + dt [rad]

function M = mean_anomaly(G, Me, a, Mo, dt)

    ===== your code goes here ====

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "dt = %f, M = %f\n", dt, mod(M, 2 * pi));
    end
end
```

Your results will be printed to the debugging file only if the global “fd” file descriptor exists. This allows enabling/disabling printing at will.

All the mathematical expressions that you need to compute “M”, given the input variables, can be found in the course slides. Remember that the Mean Anomaly INCREASES LINEARLY proportionally to the elapsed time from the ToA (M=Mo at the ToA) and to the average angular speed of the satellite (“mean motion” in [rad/s]).

3.3 Step 3

Write a matlab script (following the given prototype) to compute the current Eccentric Anomaly (in radians) of the satellite.

```
% Solve Keppler equation for Eccentric Anomally iteratively
% Loop is exited when we reach the desired tolerance
% e = orbit eccentricity
% tol = convergence tolerance
% M = Mean Anomaly [rad]
% E = eccentric anomaly [rad]

function E = eccentric(e, tol, M)

    ===== your code goes here ====

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "E = %f\n", mod(E, 2 * pi));
    end
end
```

Writing this function entails solving the Kepler equation using the method explained in the course slides. You can easily test if the result is correct by computing the expression M=E-e·sin(E) , where E is your result, and checking if you get the original value of M (modulus 2*pi).

3.4 Step 4

Write a matlab script (following the given prototype) to compute the current True Anomaly (in radians) of the satellite.

```
% compute True Anomaly [rad] given the Eccentric Anomaly [rad] and the
% orbit eccentricity
% e = orbit eccentricity
% E = Eccentric Anomaly [rad]
% v = True Anomaly [rad]

function v = true_anomaly(e, E)

    ===== your code goes here ====

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "v = %f\n", mod(v, 2 * pi));
```

```
    end
end
```

Again, all the mathematical expressions that you need to compute “v” given the input variables can be found in the course slides.

3.5 Step 5

Write a matlab script (following the given prototype) to compute the current Cartesian orbital coordinates (in meters) of the satellite.

```
% Compute cartesian coordinates of sat. within the orbital plane with
% origin at Earth center and "x" axis pointing to the ascending node
% v = true anomaly [rad]
% w = argument of perigee [rad]
% a = semi-major axis [m]
% e = orbit eccentricity
% E = Eccentric Anomaly [rad]
% [xp, yp] = cartesian orbital coordinates [m]

function [xp, yp] = orbitcoords(v, w, a, e, E)

    ===== your code goes here ====

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "xp = %f, yp = %f\n", xp, yp);
    end
end
```

All the mathematical expressions that you need to compute “xp, yp” given the input variables can be found in the course slides.

3.6 Step 6

Write a matlab script (following the given prototype) to compute the current ECEF coordinates (in meters) of the satellite.

```
% Compute CURRENT sat. ECEF coordinates [m]
% Oe = Earth rotation angular velocity (rad/s)
% Oo = longitude of AN at ToA [rad] (uncorrected)
% dt = seconds elapsed after ToA (negative if ToA is in the future)
% dO = Rate of Right Ascension of AN [rad/s]
% io = orbit incliination [rad]
% [xp, yp] = current cartesian orbital coordinates
% ecef = (1x3) vector containing the current ecef coordinates of sat.

function ecef = satecef(Oe, Oo, dO, dt, io, xp, yp)

    ===== your code goes here ====

    ecef = [x, y, z];

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "O = %f, x = %f, y = %f, z = %f\n", mod(O, 2 * pi), x, y, z);
    end
end
```

Check the course slides to find out the expressions that you need for this step. The expressions depend on the current longitude of the sat. AN.

To compute correctly the current longitude of the sat. AN you must first correct the input parameter of the function “Oo” to take into account that the AN of GPS satellites shows a small drift, i.e. the AN [rad] INCREASES linearly proportional to the elapsed time from the ToA [s] and to its rate (parameter “dO” [rad/s]).

Once you have obtained the corrected value of “Oo” at the ToA you may compute the CURRENT longitude of the AN using that the longitude of the AN of any satellite DECREASES proportionally to the elapsed time at a rate of Oe [rad/s].

Call “O” to the obtained CURRENT longitude of the AN so that it will be written to the debugging file.

Next simply apply the formulas in the course slides to obtain the ECEF coordinates x, y and z in meters.

3.7 Step 7

Once you have computed the current satellite ECEF coordinates you may use the matlab native function “ecef2lla()” to get the geodetic coordinates (LLA) of the satellite.

The sub-satellite geodetic coordinates are the same as those of the satellite setting the altitude to zero meters. The function “ecef2lla()” returns [lat, long] in degrees.

The “main1.m” main program will then plot the computed position of the sub-satellite point on top of the world map. You can check if the result is correct by opening the URL https://www.nstb.tc.faa.gov/rt_waassatellitestatus.htm. Alternatively, you can use the GNNS tools provided at http://gnssmissionplanning.com or https://in-the-sky.org/satmap_worldmap.php.

You must obtain a plot similar to this:

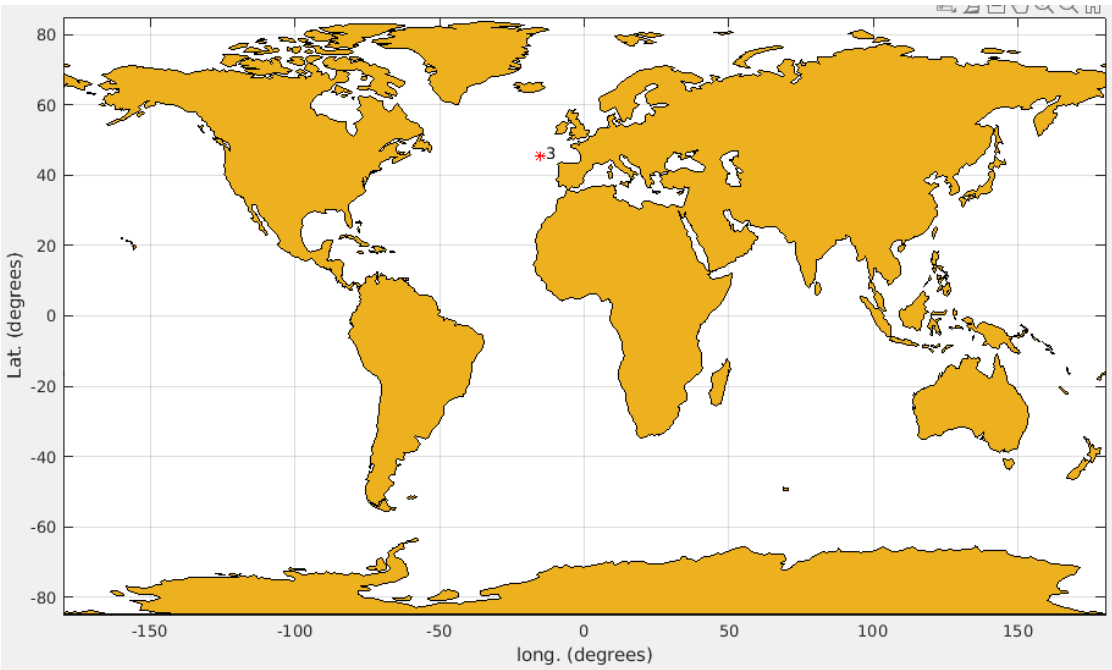

Figure 1. GPS sub-satellite position

3.8 Step 8

Write a new matlab script (following the given prototype). The script should return the current geodetic coordinates (lat, long in degrees) of the sub-satellite point.

```
% Compute (lat, long) [degrees] of sub-satellite point, given orbital
% parameters, at time = ToA + dt

% Oe = Earth rotation angular velocity (rad/s)
% G = Gravitational constant
% Me = Earth Mass
% a = orbit semi-major axis [m]
% Mo = Mean Anomaly at ToA [rad]
% dt = seconds elapsed after ToA (negative if ToA is in the future)
% e = orbit eccentricity
% tol = tolerance for convergence
% w = argument of perigee [rad]
```

```
% Oo = longitude of AN at ToA [rad] (uncorrected)
% dO = Rate of Right Ascension of AN [rad/s]
% io = orbit incliination [rad]
% long = current sub-satellite long [deg]
% lat = current sub-satellite lat [deg]

function [long, lat] = subsatellite(Oe, G, Me, a, Mo, dt, e, tol, w, Oo, dO, io)

    ===== your code goes here ====

    if ismember("fd", who("global"))
       global fd;
       fprintf(fd, "long = %f, lat = %f\n\n", long, lat);
    end
end
```

The objective of this step is to DELETE all the previously written code (after the first "fprintf") in the main program) and group it into the "subsatellite()" function.

To write this function you must simply call the previously written functions in the right order.

3.9 Step 9

In the "main1.m" program, after plotting the sub-satellite point position at the current time, write a loop that increases the variable "dt" in steps of 5 minutes and re-computes and plots the new sub-satellite position (just call the "subsatellite()" function with the new "dt").

The loop must plot the complete groundtrack of your GPS satellite. You must obtain a plot similar to this:

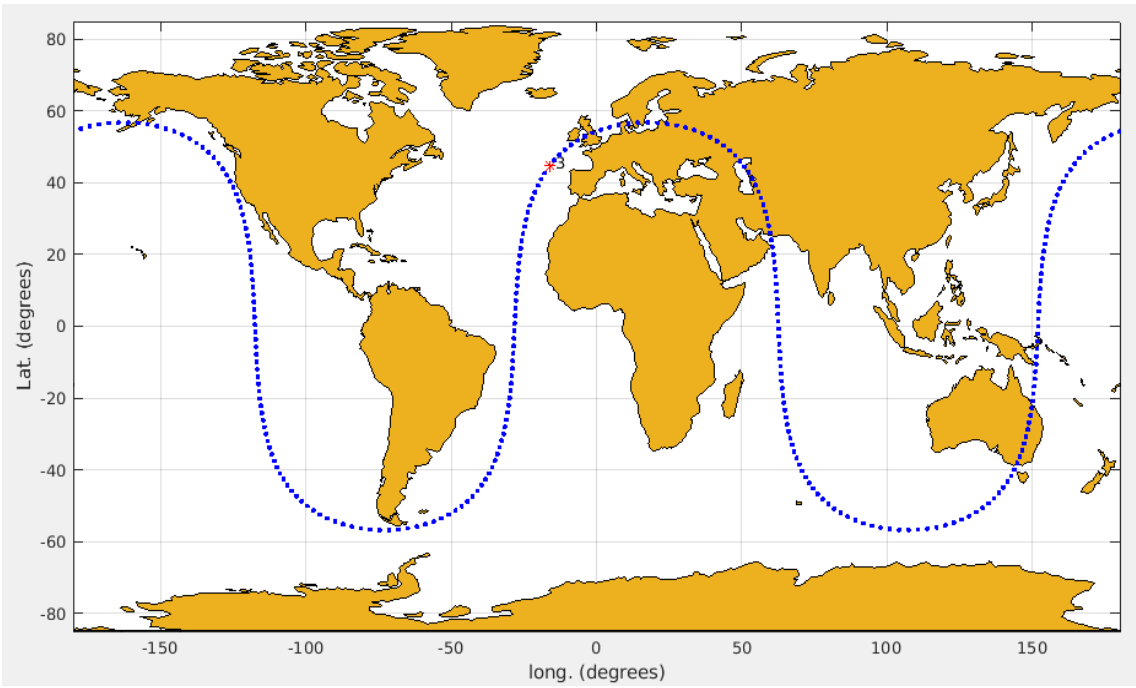

Figure 1. GPS satellite complete ground track

To accomplish the right result you must find out how many steps of the loop are necessary.

4 Final remarks

Write a report, including the screen captures of the groundtracks that you have obtained and a screen capture of the online tool at

https://www.nstb.tc.faa.gov/rt_waassatellitestatus.htm to show that your results compare well.

Explain the method and FORMULAS that you have used to perform each of the proposed 9 steps. You can print the report and use hand-written formulas if it is easier. It is advisable to write the report while the code is being developed.

The results of the laboratory work will be delivered through a repository in Atenea after the Thursday class.

Just make a zip file including the report, the code of ALL the matlab functions that you have written (as ".m" files, in order that they can be run on any computer to test its validity).

Also include in the zip the debugging file "main1.dat".

5 References

1. U.S. Department of Homeland Security. *GPS NANUS, ALMANACS, & OPS ADVISORIES.* [Online]

https://www.navcen.uscg.gov/gps-nanus-almanacs-opsadvisories-sof

2. *Definition of YUMA almanac.* [Online]

https://navcen.uscg.gov/sites/default/files/pdf/gps/ICD_GPS_240C.pdf